xCurl 1.1.0

Practical manual for integration, flow and contracts

Step by step, without assuming internal knowledge of the library

Field	Value
Version	1.1.0 stable
Supported PHP	^8.5.12
Release metadata date	25/10/2026
Distribution	Composer/Packagist recommended; GitHub VCS and standalone alternatives
Source repository	https://github.com/absolutoffice/xcurl
Document	xCurl-1.1.0-Practical-manual-en.pdf
Verification basis	Current 1.1.0 source, metadata and bundled regression checks reviewed on 25/10/2026

Who this manual is for

For users who need to install, integrate, operate or troubleshoot xCurl without having to infer its behavior from the implementation. Maintainer-only historical QA claims are not presented as current release evidence.

Absolut Office - user documentation

0. If you only read five minutes

The whole xCurl model in one page.

Main mental rule

xCurl has two layers. A TRANSPORT performs exactly one HTTPS request. HttpClient decides redirects, retries, Retry-After and the whole-operation deadline. HttpTransportFactory only selects a backend from the capabilities available at runtime.

Question	Short answer
How do I install it?	Composer through Packagist: composer require absolutoffice/xcurl:^1.1.
What API should I normally use?	HttpClient::send(HttpRequest, RequestOptions).
How is the backend selected?	HttpTransportFactory::create() or createStrict().
Does it fail over after a request failure?	No. Fallback happens only during selection by runtime capability.
What is retried by default?	GET/HEAD; transient transport failures, 408, 429 and 5xx; at most two retries.
Who follows redirects?	Only HttpClient. A transport returns 3xx without following it.
Are null and an empty request body the same?	No. null = no entity; empty string = explicit zero-byte entity.
How do I download to a file?	Use RequestOptions(targetPath: ...). The target must not already exist.
How do I diagnose the selected backend?	HttpTransportFactory::diagnostics().
How do I verify the installed runtime?	composer check-platform-reqs --no-dev, diagnostics(), then a simple HTTPS request.

Manual map

Case	Why
1-4	Mental model, installation, first request and transport selection.
5-8	Request/response contracts, downloads and typed errors.
9-13	Retries, redirects, timing, TLS/limits and diagnostics.
14-16	Current verification scope, Composer distribution and public API map.
17-22	Recipes, anti-patterns, troubleshooting, deployment, maintenance and references.
Appendix	One-page cheat sheet.

1. Mental model: what each layer does

Decide first whether you need one low-level request or a logical operation with policy.

Concept	What it means in xCurl	What it does NOT mean
Fallback	Select cURL, HTTPS streams or socket according to current runtime capabilities.	Trying another backend after DNS/TLS/read failure.
Retry	Repeat a logical request according to RetryPolicy, using the same selected transport.	Automatically changing transport.
Redirect	HttpClient follows Location and applies method/body/credential rules.	The transport following redirects on its own.

Phrase to remember

Factory chooses the engine. Transport performs one attempt. HttpClient organizes the operation.

Case

Case	Use	Why
Single request, no retry/redirect policy	HttpTransportInterface::request()	Small, predictable transport contract.
Normal application use	HttpClient::send()	Separates request data from execution policy and preserves history.
Legacy/convenience scalar call	HttpClient::request()	Compatibility wrapper over send().
Deterministic backend diagnostics/tests	HttpTransportFactory::createNamed()	Not the normal business selection path.

2. Installation and runtime

Composer is the recommended consumer path in 1.1.0. Standalone loading remains available as a compatibility fallback.

Recommended: Composer / Packagist

Install the 1.1 line in the consuming project. Composer enforces the PHP requirement and provides PSR-4 autoloading.

```
composer require absolutoffice/xcurl:^1.1
```

Direct GitHub VCS alternative

Use this when you deliberately want the stable GitHub tag as a VCS repository rather than the default Packagist resolution.

```
composer config repositories.xcurl vcs https://github.com/absolutoffice/xcurl.git
composer require absolutoffice/xcurl:1.1.0 --no-plugins --no-scripts
```

Application bootstrap

Load the consuming project autoloader once. Do not run a separate Composer installation inside vendor/absolutoffice/xcurl.

```
require_once __DIR__ . '/vendor/autoload.php';

use AbsolutOffice\XCurl\HttpTransportFactory;

$transport = HttpTransportFactory::create();
```

Production note

A production server needs the deployed vendor tree and its generated autoloader. The Composer executable itself can stay in CI/build if dependencies are installed before deployment.

Standalone fallback

The repository still includes a lightweight local autoloader. Use it only when Composer deployment is not suitable; the standalone loader does not perform Composer platform checks.

```
require_once '/path/to/xcurl/autoload.php';
```

3. First request: recommended path

Use HttpClient when the operation may need redirects, retries, a deadline or attempt history.

```
use AbsolutOffice\XCurl\Client\HttpClient;
use AbsolutOffice\XCurl\Client\RequestOptions;
use AbsolutOffice\XCurl\HttpRequest;
use AbsolutOffice\XCurl\HttpTransportFactory;

$client = new HttpClient(HttpTransportFactory::create());
$result = $client->send(
    new HttpRequest(
        'GET',
        'https://example.com/data.json',
        ['User-Agent: MyProject/1.0'],
    ),
    new RequestOptions(
        timeoutSeconds: 30,
        deadlineSeconds: 90,
    ),
);

$response = $result->response;
```

What happens

1. The factory inspects the current SAPI and creates one transport.
2. HttpClient receives immutable request data and execution options.
3. The transport validates method, URL, headers, body, limits and target path where possible before network I/O.
4. The selected transport performs one HTTPS request.
5. HttpClient decides whether to return, redirect or retry.
6. The result includes the final response, final URL, counters and typed history.

HTTP response != business success

A 404, 409 or 500 is still an HTTP response. xCurl does not throw only because the status is 4xx/5xx; retry policy may repeat selected statuses, and your application decides the final business meaning.

4. Transport selection and fallback

Selection depends on available capabilities, not on whether a particular request later fails.

Mode	Selection order	Use when
create()	cURL -> HTTPS streams -> TLS socket	General compatibility; default path.
createStrict()	cURL -> TLS socket	When xCurl itself must enforce stricter response-framing checks.

No per-request failover

If create() selects cURL and the request later fails because of DNS, timeout, TLS or a read error, xCurl does not silently try streams or socket. HttpClient may retry the same transport when policy permits.

Symptom

```
$diag = AbsolutOffice\XCurl\HttpTransportFactory::diagnostics();
print_r($diag);
```

5. Request contract: what goes in

All built-in transports share the same request validation so the same input is accepted or rejected consistently.

Field	Rule	Example
Method	Must be a valid HTTP token; normalized to uppercase; CONNECT is rejected.	get -> GET; CONNECT -> INVALID_REQUEST
HEAD	An explicit request body is rejected, including an empty string.	HEAD + body "" -> INVALID_REQUEST
URL	Absolute HTTPS only; ASCII RFC 3986 URI syntax; raw non-ASCII must be percent-encoded; URL userinfo is rejected.	https://example.com/a%20b
Headers	No unsafe controls; header name must be a token; duplicate Host is rejected.	Host may appear once and must be a valid authority.
Transfer-Encoding	Request Transfer-Encoding is not supported.	Use exact Content-Length framing.
Content-Length	If supplied, it must appear once and exactly match body bytes; otherwise xCurl synthesizes it for an explicit body.	"abc" -> 3 bytes

Body semantics

Value	Meaning	Content-Length
null	No request entity.	No synthesized Content-Length; caller may only declare 0.
""	Explicit zero-byte entity.	Content-Length: 0 is synthesized if absent.
"abc"	Binary-safe 3-byte entity.	Content-Length: 3 is synthesized if absent.

Request limit

maxRequestBytes is optional. Bodies exceeding it fail before network I/O with non-transient TransportError::REQUEST_LIMIT. A configured limit below 1 byte is invalid.

```
$transport = HttpTransportFactory::create(
    maxResponseBytes: 64 * 1024 * 1024,
    maxRequestBytes: 8 * 1024 * 1024,
);
```

6. Response contract: what comes out

Every built-in transport returns a normalized HttpResponse.

Field	Meaning
statusCode	Final HTTP status, 100..599.
headers	Final normalized response block; trailers may be present when exposed.
body	In-memory body when no target file is used.
bytes	Number of response-body bytes received or written.
targetCreated	True only when that completed attempt exclusively created the target file.

No-body semantics

HEAD and status 204, 205 and 304 are treated as responses without a body. 101 Switching Protocols is unsupported. A response ending only in an informational 1xx status is a protocol error.

Strict framing

Case	Meaning
Transfer-Encoding + Content-Length	Rejected as ambiguous.
Transfer-Encoding other than chunked	Rejected.
Conflicting Content-Length	Rejected.
Body shorter than Content-Length	Transient READ error.
PHP HTTPS streams	Compatibility backend; native wrapper may hide Transfer-Encoding, so createStrict() excludes it.

7. File downloads and ownership-safe cleanup

1.1.0 makes download ownership explicit. The target is not an overwrite destination.

```
$tmp = sys_get_temp_dir() . '/download-' . bin2hex(random_bytes(8)) . '.tmp';

$result = $client->send(
    new HttpRequest('GET', 'https://example.com/file.dat'),
    new RequestOptions(
        timeoutSeconds: 30,
        targetPath: $tmp,
        deadlineSeconds: 120,
    ),
);

// Validate size/checksum/business rules, then promote the file.
```

- When using HttpClient, targetPath must not already exist; existing files and symlinks are refused before the first request.
- Built-in transports create body targets exclusively. No-body responses do not create a target.
- On a transport failure, a partial file created by that transport is removed by the transport.
- Before a redirect or response retry, HttpClient removes only an intermediate file whose completed response reports targetCreated=true.
- If a file appears but the completed attempt did not claim ownership, continuation stops with ClientError::TARGET_CLEANUP and the file is not deleted.
- Use a fresh temporary path for every operation; validate checksum/size/business rules and then rename/promote it in application code.

8. Errors: how to know what happened

Do not parse exception messages for control flow. Use enums, transient and client history.

Transport errors

TransportError	value
DNS	dns
CONNECT	connect
TLS	tls
TIMEOUT	timeout
READ	read
WRITE	write
PROTOCOL	protocol
RESPONSE_LIMIT	response_limit
REQUEST_LIMIT	request_limit
INVALID_REQUEST	invalid_request
UNAVAILABLE	unavailable
UNKNOWN	unknown

Client errors

ClientError	Meaning
INVALID_REDIRECT	Multiple/invalid/non-HTTPS redirect target.
REDIRECT_LIMIT	Maximum followed redirects exceeded.
DEADLINE	Not enough time remains for redirect, retry or another attempt.
TARGET_CLEANUP	Safe continuation cannot proceed because an intermediate target cannot be safely removed/owned.
TRANSPORT	Transport failure not retried or retry conditions exhausted.

transient matters

TransportError classifies the category. HttpTransportException::$transient says whether retrying the same transport may reasonably succeed. RetryPolicy additionally checks method and retry budget.

```
try {
    $result = $client->send($request, $options);
} catch (AbsolutOffice\XCurl\Client\HttpClientException $e) {
    error_log('client_error=' . $e->error->value);
    foreach ($e->history as $attempt) {
        error_log($attempt->action->value);
    }
}
```

9. Retries: when xCurl repeats

The default policy is conservative to avoid duplicating side effects.

Parameter	Default	Effect
maxRetries	2	At most two retries after the first attempt.
backoffMilliseconds	[1000, 3000]	Delay for retry #1 and #2; last value is reused if needed.
methods	[GET, HEAD]	Only these methods are automatically retryable by default.
statusCodes	[408, 429]	Explicit retryable statuses.
retryServerErrors	true	Also retries 500..599.
maxRetryAfterSeconds	60	Cap for a valid Retry-After value.

POST is not retried by default

If you enable POST/PUT/PATCH in RetryPolicy, idempotency is your application/API responsibility. xCurl cannot know whether repeating a side-effecting request is safe.

10. Redirects: exact rules

HttpClient follows redirects manually; transports never enable automatic redirect following.

Status	Next method	Body	Entity headers
301	preserve	preserve	preserve
302	preserve	preserve	preserve
303	GET unless HEAD	drop if method changes	drop Content-Length and Content-Type if method changes
307	preserve	preserve	preserve
308	preserve	preserve	preserve

Header security rules

- Every followed redirect removes an explicit Host so the next authority is regenerated.
- Cross-origin redirects strip Authorization by default.
- Cross-origin redirects strip Cookie by default.
- Both cross-origin stripping behaviors are configurable in RedirectPolicy.
- Multiple Location fields are invalid while following is enabled.
- maxRedirects: 0 means do not follow redirects; the 3xx response is returned unchanged.
- Redirect continuation remains HTTPS-only.

11. Timeout and deadline are different

timeoutSeconds protects one transport attempt; deadlineSeconds caps the whole HttpClient operation.

Concept	What it means in xCurl	Effect
timeoutSeconds	One transport attempt.	Safety timeout; backend-specific semantics.
deadlineSeconds	Whole HttpClient operation.	Includes retries, redirects and sleeps; caps each new attempt.

Backend

Backend	timeoutSeconds behavior
cURL	Total request timeout and connect cap.
HTTPS streams	Native wrapper I/O timeout plus timed_out check after reads.
TLS socket	TCP connect, TLS handshake I/O and subsequent stream I/O.

Deadline behavior

Before each new attempt HttpClient computes remaining time. If less than one second remains, or a configured delay plus another attempt does not fit, it fails with ClientError::DEADLINE.

12. TLS, private CA and size limits

TLS peer and hostname verification remain enabled. A custom CA bundle changes trust anchors, not verification policy.

```
$caFile = __DIR__ . '/config/private-ca.pem';

$curl    = new CurlHttpTransport($caFile);
$streams = new StreamHttpTransport($caFile);
$socket  = new SocketHttpTransport($caFile);
```

Rule	Meaning
Peer verification	Enabled.
Hostname verification	Enabled.
CA path empty/NUL	Invalid configuration.
maxResponseBytes < 1	Invalid configuration.
maxRequestBytes < 1	Invalid configuration.
Response exceeds limit	TransportError::RESPONSE_LIMIT, non-transient; partial owned target removed.
Request exceeds limit	TransportError::REQUEST_LIMIT, non-transient; rejected before network I/O.

```
$transport = HttpTransportFactory::create(
    maxResponseBytes: 64 * 1024 * 1024,
    maxRequestBytes: 8 * 1024 * 1024,
);
```

13. Diagnostics: what each SAPI can use

CLI and Apache/FPM can run the same PHP version with different php.ini files, extensions and capabilities. Check diagnostics in the SAPI that actually performs the request.

```
$diag = AbsolutOffice\XCurl\HttpTransportFactory::diagnostics();
print_r($diag);
```

Symptom	Check
cURL false	Check ext-curl and required cURL functions.
streams false	Check allow_url_fopen and the https wrapper.
socket false	Check ext-openssl plus tls/ssl stream transports.
selectedStrict=socket while streams=true	Expected: streams are excluded from strict mode.

14. Verification in 1.1.0

The 1.1.0 checkout uses a smaller, explicit verification surface than the historical 1.0 development tree. Do not present historical 10/14-gate reports as evidence for 1.1.0.

Consumer verification

After installation, verify the real runtime platform, inspect transport capabilities, then run a simple HTTPS request in the target SAPI.

```
composer check-platform-reqs --no-dev
```

```
$diag = AbsolutOffice\XCurl\HttpTransportFactory::diagnostics();
print_r($diag);
```

Checkout verification

From the repository root, the current installation guide documents these reproducible checks:

```
composer validate --strict --no-check-lock
composer test
php tests/composer-consumer.php
find src tests -name '*.php' -print0 | xargs -0 -n1 php -l
php -l autoload.php
git diff --check
```

What these checks prove

- composer validate checks package metadata syntax/consistency.
- composer test runs the bundled dependency-free regression runner.
- tests/composer-consumer.php verifies PHP 8.5.11 rejection, simulated 8.5.12 Composer installation, vendor/autoload.php public-API smoke and Composer archive contents.
- Syntax checks cover current src/tests/autoload.php files.

What they do not prove

- They do not constitute the historical full cURL/streams/TLS integration suite.
- They do not run the historical fire matrix or browser QA panel; those tools are not bundled in the current checkout.
- They do not by themselves prove remote installation from GitHub or Packagist.

15. Composer distribution and package contents

1.1.0 is structured as a normal Composer library. The consumer runtime requirement is only PHP ^8.5.12; contributor tooling stays in require-dev.

- PSR-4: AbsolutOffice\XCurl\ -> src/.
- No root composer.json version field; stable versions come from VCS tags.
- Composer archives exclude local indexes, caches, vendor, environment files, PDFs and key material.
- LICENSE, COPYING, runtime source and bundled regression tests remain distributed by the current archive checks.
- PDF manuals stored in the repository are intentionally excluded from Composer archives by the current export/archive rules.

```
"require": {
    "php": "^8.5.12"
},
"autoload": {
    "psr-4": {
        "AbsolutOffice\\XCurl\\": "src/"
    }
}
```

16. Public API map for the 1.x line

The 1.0 API reference remains the historical compatibility baseline for the 1.x line. The current 1.1.0 source keeps the same core public types and adds ownership reporting through HttpResponse::targetCreated.

Type	Role	Remember
HttpTransportFactory	Factory/diagnostics	create, createStrict, createNamed, diagnostics, selected*
HttpTransportInterface	Transport contract	request(...) -> HttpResponse
CurlHttpTransport	Concrete transport	cURL
StreamHttpTransport	Concrete transport	PHP HTTPS streams
SocketHttpTransport	Concrete transport	Direct TLS socket
HttpRequest	Value object	method, url, headers, body
HttpResponse	Value object	statusCode, headers, body, bytes, targetCreated
HttpTransportException	Transport exception	transient + TransportError
TransportError	Transport enum	12 stable categories
Client\HttpClient	Policy orchestrator	send() preferred; request() wrapper
Client\RequestOptions	Execution options	timeoutSeconds, targetPath, deadlineSeconds
Client\RetryPolicy	Retry policy	methods, statuses, backoff, Retry-After
Client\RedirectPolicy	Redirect policy	limit + cross-origin credential stripping
Client\HttpClientResult	Final result	response, finalUrl, history, redirects, retries
Client\HttpClientAttempt	Attempt record	sequence, method, url, action, status/error/delay/nextUrl
Client\HttpClientAction	History enum	RETURNED, REDIRECTED, RETRIED, FAILED
Client\HttpClientException	Client exception	ClientError + history
Client\ClientError	Client enum	5 categories
ClockInterface	Extension/test contract	monotonicSeconds, unixTime
SleeperInterface	Extension/test contract	sleepMilliseconds

Internal surface

Internal*, RequestValidator, SystemClock/SystemSleeper and transport capability helpers are implementation/test details; normal consumers should not build business integration around them.

17. Copyable recipes

Small patterns for common cases.

A. Simple GET

Use HttpClient + HttpRequest + RequestOptions and inspect status/header/body.

```
require_once __DIR__ . '/vendor/autoload.php';

use AbsolutOffice\XCurl\HttpTransportFactory;

$transport = HttpTransportFactory::create();
$response = $transport->request(
    'GET',
    'https://example.com/',
    [],
    30,
);

echo $response->statusCode, PHP_EOL;
```

B. POST JSON without automatic retry

POST is not retryable by default; pass a raw JSON body and Content-Type.

```
$payload = json_encode($data, JSON_THROW_ON_ERROR);
$result = $client->send(
    new HttpRequest(
        'POST',
        'https://example.com/api/items',
        ['Content-Type: application/json'],
        $payload,
    ),
    new RequestOptions(timeoutSeconds: 30, deadlineSeconds: 60),
);
```

C. Binary PUT with limits

Use createStrict(maxResponseBytes: ..., maxRequestBytes: ...) when strict framing is required.

```
$transport = HttpTransportFactory::createStrict(
    maxResponseBytes: 4 * 1024 * 1024,
    maxRequestBytes: 16 * 1024 * 1024,
);
```

D. Large download to temporary file

Use a unique targetPath; validate and promote after success.

```
$tmp = sys_get_temp_dir() . '/download-' . bin2hex(random_bytes(8)) . '.tmp';

$result = $client->send(
    new HttpRequest('GET', 'https://example.com/file.dat'),
    new RequestOptions(
        timeoutSeconds: 30,
        targetPath: $tmp,
        deadlineSeconds: 120,
    ),
);

// Validate size/checksum/business rules, then promote the file.
```

E. Disable redirect following

Construct RedirectPolicy(maxRedirects: 0).

```
$client = new HttpClient(
    HttpTransportFactory::create(),
    redirectPolicy: new RedirectPolicy(maxRedirects: 0),
);
```

F. Retry POST only when your API is safe

Add POST to RetryPolicy methods only with a real idempotency guarantee.

```
$retry = new RetryPolicy(maxRetries: 1, methods: ['GET', 'HEAD', 'POST']);
$client = new HttpClient(HttpTransportFactory::create(), $retry);
```

18. Anti-patterns

Avoid these integration mistakes.

Do not	Do instead
“If cURL fails, manually try streams.”	Keep the selected transport; let HttpClient retry only if the same-transport failure is transient and policy allows it.
Put provider URLs/credentials/business rules inside xCurl.	Keep them in the consuming application.
Enable backend-native follow redirects.	Leave redirects to HttpClient for consistent policy.
Send request Transfer-Encoding.	Send a normal raw body; xCurl uses exact Content-Length framing.
Decide logic from exception text.	Use TransportError/ClientError, transient and history.
Use createNamed() as normal business selection.	Use create()/createStrict(); createNamed() is for deterministic diagnostics/tests.
Reuse or overwrite an existing download target.	Use a new temporary path for each operation.
Expect timeout=30 to mean exactly 30.000 s on every backend.	Treat it as a safety timeout; use deadline for whole-operation budgeting.

19. Troubleshooting by symptom

Start with the symptom and diagnostics; do not change transports at random.

Symptom	Check	Why
“No usable HTTPS transport”	HttpTransportFactory::diagnostics()	Check cURL, allow_url_fopen/https wrapper, OpenSSL tls/ssl support.
Works in CLI, fails in web SAPI	Compare diagnostics in both SAPIs	Check different php.ini files and loaded extensions.
TLS failure	TransportError::TLS + CA/hostname	Check certificate SAN, chain and trust bundle.
Target is refused before request	Does the path already exist or is it a symlink?	Use a unique new temporary target.
Target cleanup stops continuation	ClientError::TARGET_CLEANUP	An unowned/intermediate file appeared or could not be safely removed.
POST is not retried	RetryPolicy methods	Default is GET/HEAD only.
Redirect not followed	RedirectPolicy maxRedirects + Location	0 disables following; Location must resolve to HTTPS.
Deadline before maxRetries	deadlineSeconds + delays	Expected when another complete attempt no longer fits.
Streams accepts framing that strict mode rejects	create() vs createStrict()	Expected: strict mode excludes streams.

20. Deployment checklist for a consumer

The normal 1.1.0 deployment is Composer-first.

1. Require absolutoffice/xcurl:^1.1 in the consuming application.
2. Deploy the application vendor tree generated by Composer.
3. Load vendor/autoload.php once from the application bootstrap.
4. Choose create() or createStrict() according to framing requirements.
5. Use HttpClient::send() unless you intentionally need a single transport call.
6. Define timeout and, for multi-attempt operations, deadline.
7. Define request/response limits when the use case needs them.
8. Keep credentials, endpoints, checksums, schemas and business rules outside xCurl.
9. For downloads, use a fresh temporary target and validate before final promotion.
10. Log typed error enums and history instead of depending on message text.
11. Optionally run composer check-platform-reqs --no-dev on the real deployment runtime.

Production note

A production server needs the deployed vendor tree and its generated autoloader. The Composer executable itself can stay in CI/build if dependencies are installed before deployment.

21. Compatibility and maintenance

xCurl 1.1.0 requires PHP ^8.5.12. The 1.0 reference remains the documented compatibility baseline for the 1.x line, while 1.1.0 adds backward-compatible distribution and download-ownership behavior.

Change	Treatment
Internal bugfix without public break	May enter patch/minor after current checks and compatibility review.
Backward-compatible public addition	Minor release is appropriate when behavior/API expands compatibly.
Change that breaks existing public signatures/semantics	Requires an explicit semantic-versioning decision.
Internal/test helper change	Not a consumer contract, but regression checks should still pass.
Historical QA references	Keep as archival context; do not reuse them as current 1.1.0 validation evidence.

22. Which source wins when there is doubt

This manual explains. Current source and release/package metadata define the behavior of the 1.1.0 checkout.

Source	Use
src/	Exact runtime implementation.
composer.json	Package name, PHP requirement, PSR-4, dev tooling and archive rules.
README.md	Compact operational overview.
docs/INSTALLATION.md	Installation paths and current verification commands.
docs/releases/1.1.0.md	Release-specific changes and validation limits.
tests/regression.php	Bundled dependency-free behavior checks.
tests/composer-consumer.php	Composer consumer/archive checks.
docs/reference/1.0.0/API-1.0.0.md	Historical 1.0 public API compatibility reference for the 1.x line.

Closing rule

Keep the layers separate: the application owns business rules; HttpClient owns policy; HttpTransportInterface performs one request; HttpTransportFactory selects the available backend.

Appendix A. One-page cheat sheet

I need...	Do...
Install	composer require absolutoffice/xcurl:^1.1
Normal use	new HttpClient(HttpTransportFactory::create()) + send(HttpRequest, RequestOptions)
Strict framing	HttpTransportFactory::createStrict()
One request without policy	$transport->request(...)
Download	RequestOptions(targetPath: $tmp), where $tmp does not exist
Raw body	HttpRequest(..., body: $payload); null != ""
Size limits	create(maxResponseBytes: ..., maxRequestBytes: ...)
Retries	RetryPolicy; default GET/HEAD, maxRetries=2
Redirects	RedirectPolicy; default maxRedirects=5
Whole-operation budget	RequestOptions(deadlineSeconds: ...)
Diagnose backend	HttpTransportFactory::diagnostics()
Understand transport failure	HttpTransportException: error + transient
Understand client failure	HttpClientException: error + history
Verify runtime	composer check-platform-reqs --no-dev
Current checkout regression	composer test + php tests/composer-consumer.php

Selection order

Mode	1	2	3
create()	cURL	streams	socket
createStrict()	cURL	socket	-

Redirects

301	302	303	307	308
preserve	preserve	GET except HEAD; drop body if method changes	preserve	preserve